CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
DOUTORADO EM MODELAGEM MATEMÁTICA COMPUTACIONAL
ALGORITMOS E ESTRUTURAS DE DADOS

COMPARAÇÃO DOS MÉTODOS DE ORDENAÇÃO QUICKSORT E HEAPSORT

Leonardo Vieira Guimarães

Belo Horizonte
2025

Sumário

1 Introdução **2**

2 Desenvolvimento **2**

2.1 Experimentos e Plotagem dos Resultados 3

2.1.1 Resultados para Vetores Ordenados em Ordem Crescente (Melhor Caso) 3

2.1.2 Resultados para Vetores Ordenados em Ordem Decrescente (Pior Caso) 5

2.1.3 Resultados para Vetores Aleatórios (Caso Médio) 6

3 Conclusão **8**

1 Introdução

A ordenação de dados é uma das tarefas fundamentais em Ciência da Computação, sendo essencial para otimizar buscas, análises e diversas operações em estruturas de dados. Entre os algoritmos clássicos de ordenação, destacam-se o Quicksort e o Heapsort, ambos amplamente utilizados devido à sua eficiência e aplicabilidade em diferentes contextos.

O objetivo deste trabalho é realizar uma análise comparativa entre os algoritmos Quicksort (utilizando sempre o primeiro elemento como pivô) e Heapsort, com foco no número de comparações realizadas durante a ordenação de vetores em diferentes cenários: vetores já ordenados em ordem crescente (melhor caso), vetores ordenados em ordem decrescente (pior caso) e vetores com elementos aleatórios (caso médio). A partir dessa análise, busca-se compreender o impacto da escolha do algoritmo e das características da entrada no desempenho observado, destacando vantagens, limitações e situações em que cada método se mostra mais eficiente.

2 Desenvolvimento

Para atender ao objetivo do trabalho, foram seguidos os seguintes passos:

1. **Implementação dos métodos de ordenação:**
   - **Quicksort:** pivô sempre o primeiro elemento do vetor.
   - **Heapsort:** construção do heap máximo e ordenação por remoção sucessiva do maior elemento.
2. **Testes realizados para cada método:**
   a) Gerar vetores de n elementos **ordenados em ordem crescente**, com n variando de 1000 até 10.000, com intervalo de 1000. Ordenar esses vetores com ambos os métodos e computar o número de comparações realizadas.
   b) Gerar vetores de n elementos **ordenados em ordem decrescente**, com n variando de 1000 até 10.000, com intervalo de 1000. Ordenar esses vetores com ambos os métodos e computar o número de comparações realizadas.
   c) Gerar vetores de n elementos **aleatórios**, com n variando de 1000 até 10.000, com intervalo de 1000. Ordenar esses vetores com ambos os métodos e computar o número de comparações realizadas.
   d) Fazer um único gráfico de n versus número de comparações, levando em consideração todos os testes realizados.

3. **Análise dos comportamentos dos gráficos gerados**

A Tabela 1 apresenta as fórmulas de recorrência e as ordens de complexidade assintótica dos algoritmos Quicksort e Heapsort para os diferentes cenários (melhor caso, caso médio e pior caso) analisados neste trabalho, nesta tabela podemos observar que ambos os algoritmos possuem complexidade $O(n \log n)$ no melhor caso e no caso médio, mas o Quicksort apresenta complexidade $O(n^2)$ no pior caso, enquanto o Heapsort mantém $O(n \log n)$.

Caso	**Quicksort**		**Heapsort**	
	T(n)	**O**	**T(n)**	**O**
Melhor	$2T(\frac{n}{2}) + cn$	$O(n \log n)$	$O(n) + nO(\log n)$	$O(n \log n)$
Médio	$2T(\frac{n}{2}) + cn$	$O(n \log n)$	$O(n) + nO(\log n)$	$O(n \log n)$
Pior	$T(n-1) + cn$	$O(n^2)$	$O(n) + nO(\log n)$	$O(n \log n)$

Tabela 1: Resumo das complexidades de Quicksort e Heapsort.

2.1 Experimentos e Plotagem dos Resultados

Para a realização dos experimentos, os algoritmos Quicksort e Heapsort foram implementados em Python, seguindo rigorosamente as especificações do enunciado. Os testes seguiram os seguintes passos:

1. Para cada valor de n no intervalo de 1000 a 10.000 (com incremento de 1000), foram gerados três tipos de vetores: ordenados em ordem crescente, ordenados em ordem decrescente e aleatórios.

2. Cada vetor foi ordenado utilizando ambos os algoritmos, e o número de comparações realizadas foi registrado.

3. Os resultados foram utilizados para gerar gráficos que relacionam o tamanho do vetor (n) ao número de comparações realizadas.

2.1.1 Resultados para Vetores Ordenados em Ordem Crescente (Melhor Caso)

Foram gerados vetores ordenados em ordem crescente, variando de 1000 a 10.000 elementos. Cada vetor foi ordenado utilizando Quicksort e Heapsort, e o número de comparações foi registrado.

O gráfico (Figura 1) mostra o número de comparações realizadas por cada algoritmo para vetores ordenados em ordem crescente:

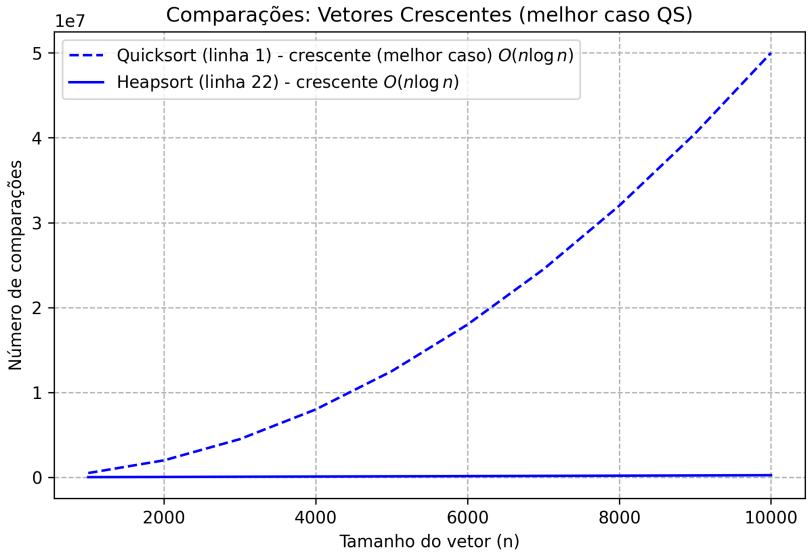

Figura 1: Número de comparações: Quicksort vs Heapsort para vetores ordenados em ordem crescente (escala linear).

Para melhor visualização das diferenças de desempenho, também foi plotado o gráfico na escala logarítmica no eixo y (Figura 2):

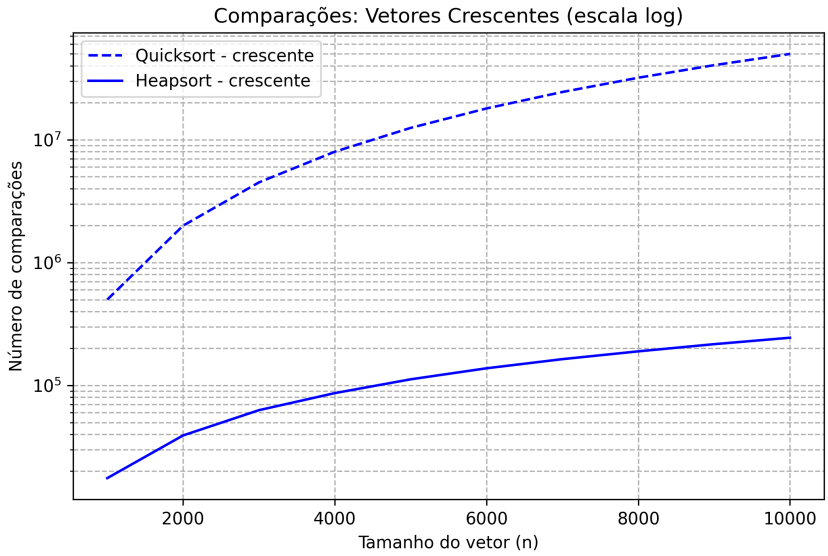

Figura 2: Número de comparações: Quicksort vs Heapsort para vetores ordenados em ordem crescente (escala logarítmica).

Apesar de ambos apresentarem complexidade assintótica $O(n \log n)$ tanto no melhor caso (vetores ordenados) quanto no caso médio (vetores aleatórios), os gráficos mostram que o Quicksort realiza um número significativamente menor de comparações do que o Heapsort nessas situações. Isso ocorre porque, além do termo dominante, os fatores

constantes e termos independentes diferem: no Quicksort, o termo constante cn é menor, enquanto no Heapsort há um termo linear adicional $O(n)$ devido à construção e manutenção do heap. Assim, na prática, o Quicksort tende a ser mais eficiente em número de comparações, mesmo com a mesma ordem de complexidade teórica.

2.1.2 Resultados para Vetores Ordenados em Ordem Decrescente (Pior Caso)

Foram gerados vetores ordenados em ordem decrescente, variando de 1000 a 10.000 elementos. Cada vetor foi ordenado utilizando Quicksort e Heapsort, e o número de comparações foi registrado.

O gráfico (Figura 3) mostra o número de comparações realizadas por cada algoritmo para vetores ordenados em ordem decrescente:

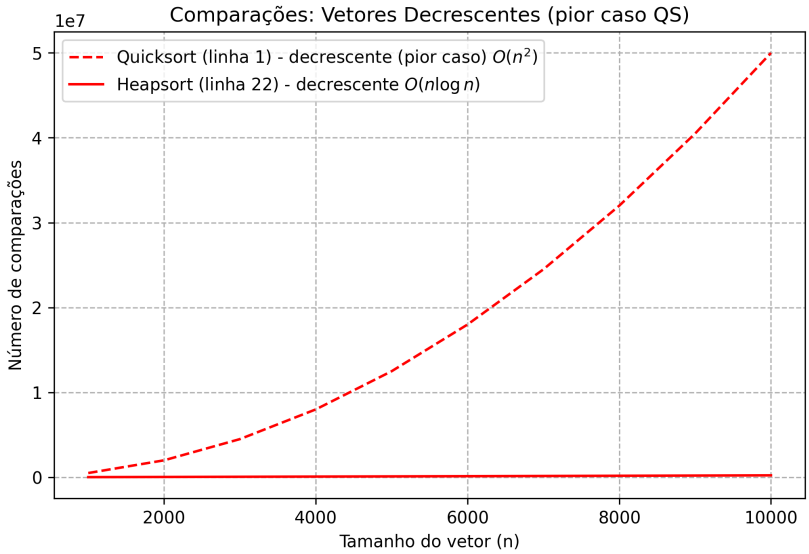

Figura 3: Número de comparações: Quicksort vs Heapsort para vetores ordenados em ordem decrescente (escala linear).

Para melhor visualização das diferenças de desempenho, também foi plotado o gráfico na escala logarítmica no eixo y (Figura 4):

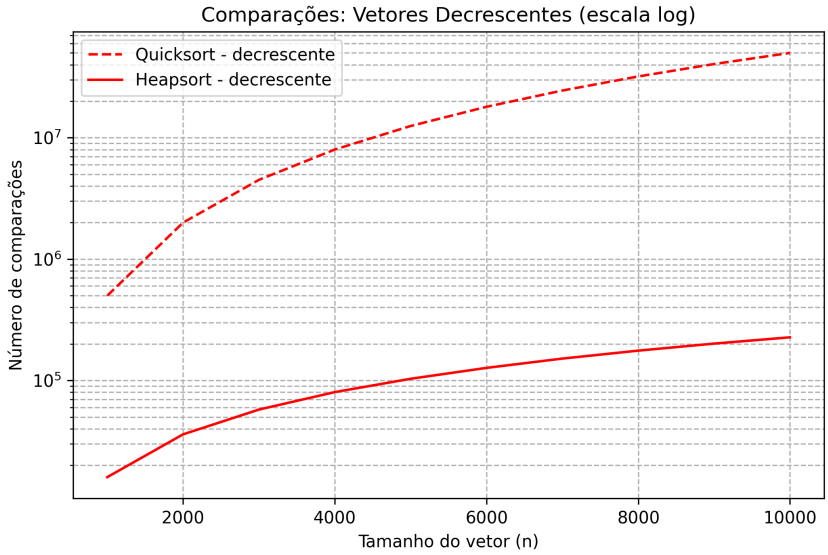

Figura 4: Número de comparações: Quicksort vs Heapsort para vetores ordenados em ordem decrescente (escala logarítmica).

No pior caso, que ocorre para o Quicksort quando o vetor está ordenado em ordem decrescente (ou crescente, com pivô fixo no início), o número de comparações cresce quadraticamente, resultando em complexidade $O(n^2)$. Isso é claramente observado nos gráficos, onde o Quicksort apresenta um crescimento muito mais acentuado no número de comparações em relação ao Heapsort. O Heapsort, por sua vez, mantém sua complexidade $O(n \log n)$ mesmo no pior caso, mostrando-se mais robusto e estável independentemente da ordem inicial dos dados.

2.1.3 Resultados para Vetores Aleatórios (Caso Médio)

Foram gerados vetores com elementos aleatórios, variando de 1000 a 10.000 elementos. Cada vetor foi ordenado utilizando Quicksort e Heapsort, e o número de comparações foi registrado.

O gráfico (Figura 5) mostra o número de comparações realizadas por cada algoritmo para vetores aleatórios:

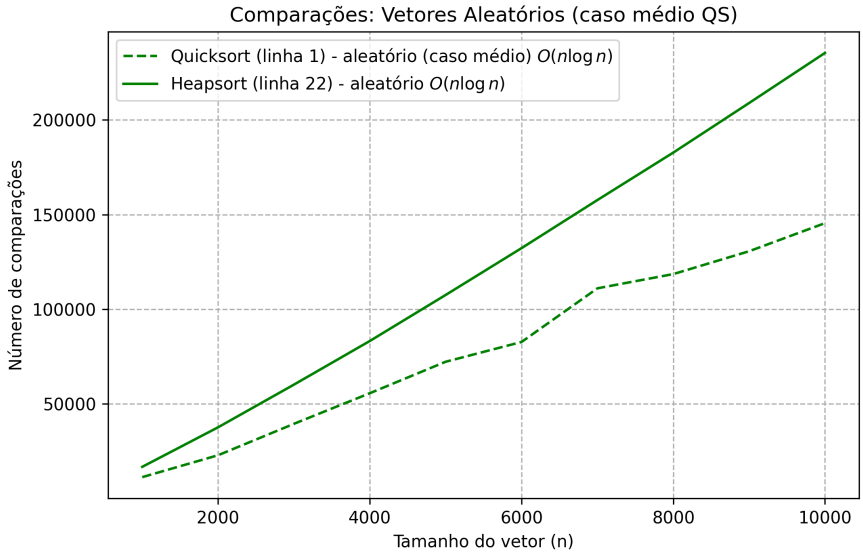

Figura 5: Número de comparações: Quicksort vs Heapsort para vetores aleatórios (escala linear).

Para melhor visualização das diferenças de desempenho, também foi plotado o gráfico na escala logarítmica no eixo y (Figura 6):

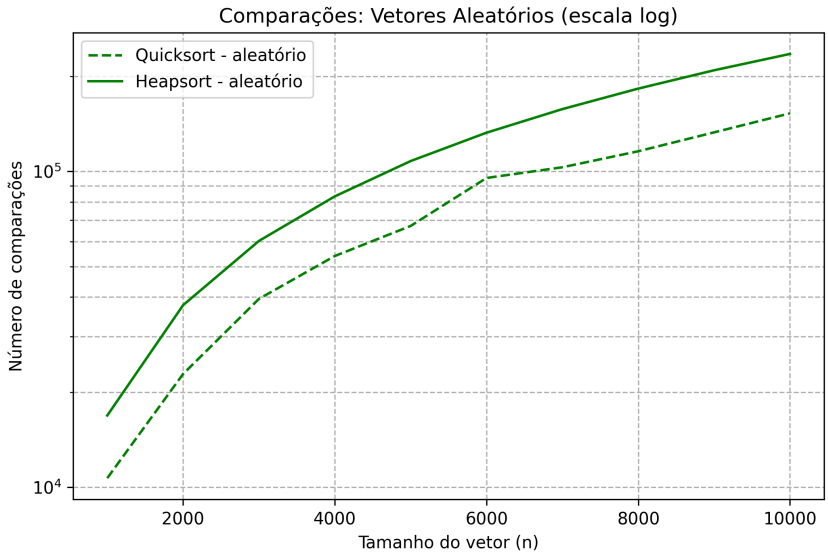

Figura 6: Número de comparações: Quicksort vs Heapsort para vetores aleatórios (escala logarítmica).

No caso dos vetores aleatórios, ambos os algoritmos mantêm a complexidade $O(n \log n)$, mas o Quicksort ainda realiza menos comparações, pois tende a dividir o vetor de forma mais equilibrada. O Heapsort, apesar de estável, apresenta um custo adicional nas operações de construção e manutenção do heap, resultando em um número de comparações um pouco maior na prática, mesmo mantendo a mesma ordem de complexidade.

Para vetores ordenados em ordem decrescente, o comportamento é semelhante ao do caso crescente: o Quicksort sofre com divisões desbalanceadas, resultando em complexidade quadrática $O(n^2)$ e um número muito elevado de comparações. O Heapsort, por outro lado, mantém desempenho estável e próximo de $O(n \log n)$, independentemente da ordem inicial dos dados. Isso evidencia que o Heapsort é mais robusto frente a diferentes tipos de entrada, enquanto o Quicksort é mais sensível à escolha do pivô e à ordem dos dados.

3 Conclusão

A análise dos resultados experimentais, ilustrados nos gráficos consolidados das Figuras 7 e 8, evidencia diferenças marcantes entre Quicksort e Heapsort conforme o tipo de entrada.

O gráfico da Figura 7 apresenta todos os cenários juntos, permitindo comparar o comportamento dos algoritmos em diferentes tipos de entrada:

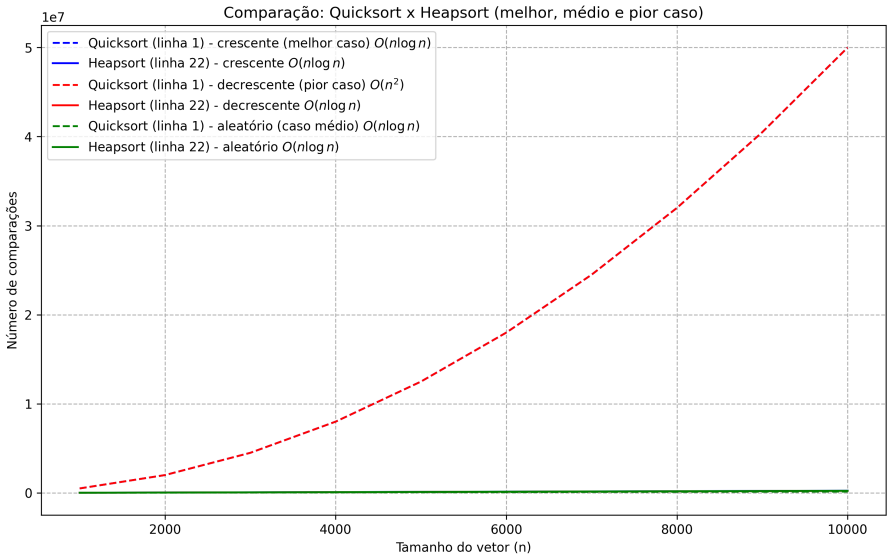

Figura 7: Número de comparações: Quicksort vs Heapsort para diferentes tipos de entrada (escala linear).

Para análise em escala logarítmica, foi gerado também o gráfico da Figura 8:

Para vetores ordenados (crescente), o Quicksort apresenta desempenho superior, realizando um número significativamente menor de comparações do que o Heapsort, mesmo ambos tendo complexidade $O(n \log n)$. Isso se deve ao menor fator constante do Quicksort, já que o Heapsort possui um termo linear adicional devido à construção do

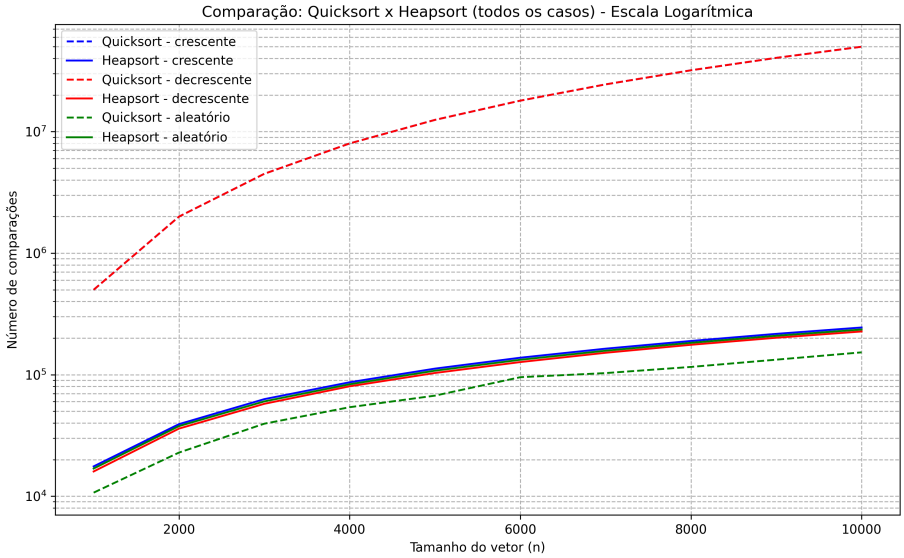

Figura 8: Número de comparações: Quicksort vs Heapsort para diferentes tipos de entrada (escala logarítmica).

heap.

No caso de vetores aleatórios, ambos os algoritmos mantêm a complexidade $O(n \log n)$, mas o Quicksort ainda realiza menos comparações, pois tende a dividir o vetor de forma mais equilibrada. O Heapsort, apesar de estável, apresenta um custo adicional nas operações de construção e manutenção do heap, resultando em um número de comparações um pouco maior na prática, mesmo mantendo a mesma ordem de complexidade.

Para vetores ordenados em ordem decrescente, o Quicksort sofre seu pior caso, com crescimento quadrático $O(n^2)$ no número de comparações, tornando-se muito menos eficiente que o Heapsort, que mantém sua robustez e estabilidade com $O(n \log n)$.

Portanto, os experimentos mostram que o Quicksort é mais eficiente em entradas aleatórias ou ordenadas, mas pode ser muito penalizado em casos desfavoráveis. O Heapsort, por sua vez, é mais estável e previsível, sendo indicado quando se deseja garantir desempenho consistente independentemente da ordem inicial dos dados.